ELEC0129 Introduction to Robotics
March 2024
Group 15

Report on Pick-and-Place Task of a Robot

Introduction ***1***

Approach ***1***

Part 1 Building of Robot ***1***

Part 2 Forward Kinematics ***3***

Calculations **4**

Experimental Results **6**

Part 3 Inverse Kinematics ***7***

Geometric Method **7**

Multiplicities **8**

Experimental Results **9**

Summary **9**

Part 4 Trajectory Planning ***10***

Experimental Results **12**

Conclusion ***12***

Appendix ***13***

The code for controlling picking up and dropping down: **13**

Introduction

The aim of our robotics project is to build a robot that can pick up an object from one position, return to a 'home' position, and then release it at another position. This will be achieved using an Arduino board controlled by code, which will move the robot according to calculations related to forward kinematics (the process of obtaining the position of the end-effector, given the known joint angles) and inverse kinematics (the process of calculating the joint angles of the robot, given a desired end-effector position). The following report will detail our calculations and interpretations for all processes relevant to this task.

Approach

Our general method is as follows:

1. Forward Kinematics: will allow us to find the position of the end effector depending on the joint angles of the robot. The main purpose of this is to help us verify the next step.
2. Inverse Kinematics: is the process of calculating the joint angles based on a desired end-effector position. The main reason for this is that we need to be able to move the end-effector to the object (given we know the object's position), pick it up, and move it to a location. In order to do this, we need to know the joint angles at all these positions, which is why inverse kinematics is needed.
    - Note: forward kinematics is useful to verify that the angles being calculated are correct. We can take the angles produced by inverse kinematics, feed it back into forward kinematics, and use that process to calculate the end-effector position to see if it matches the one inputted into inverse kinematics.
3. Trajectory Planning: this final step will allow us to make smooth movements between points. By using a "cubic polynomial" formula, the movement from start to finish will be smooth because it models a polynomial. This means that towards the start and the end, the speed will be slightly lower, and the motion will be a straight line so that the movement doesn't jerk too much.

Part 1 Building of Robot

When building the robot, angles and links are defined:

L_1 – *Distance between joint 2 and joint 3*
L_2 – *Distance between joint 3 and end effector*
θ_1 – *rotation of joint 1*
θ_2 – *rotation of joint 2 (angle between link 1 and the base)*
θ_3 – *rotation of joint 3 (angle between link 1 and link 2)*

With measurements, it is known that:

$$L_1 = 9.4cm$$
$$L_2 = 17.7cm$$

The following diagram illustrates each of these values on the actual robot.

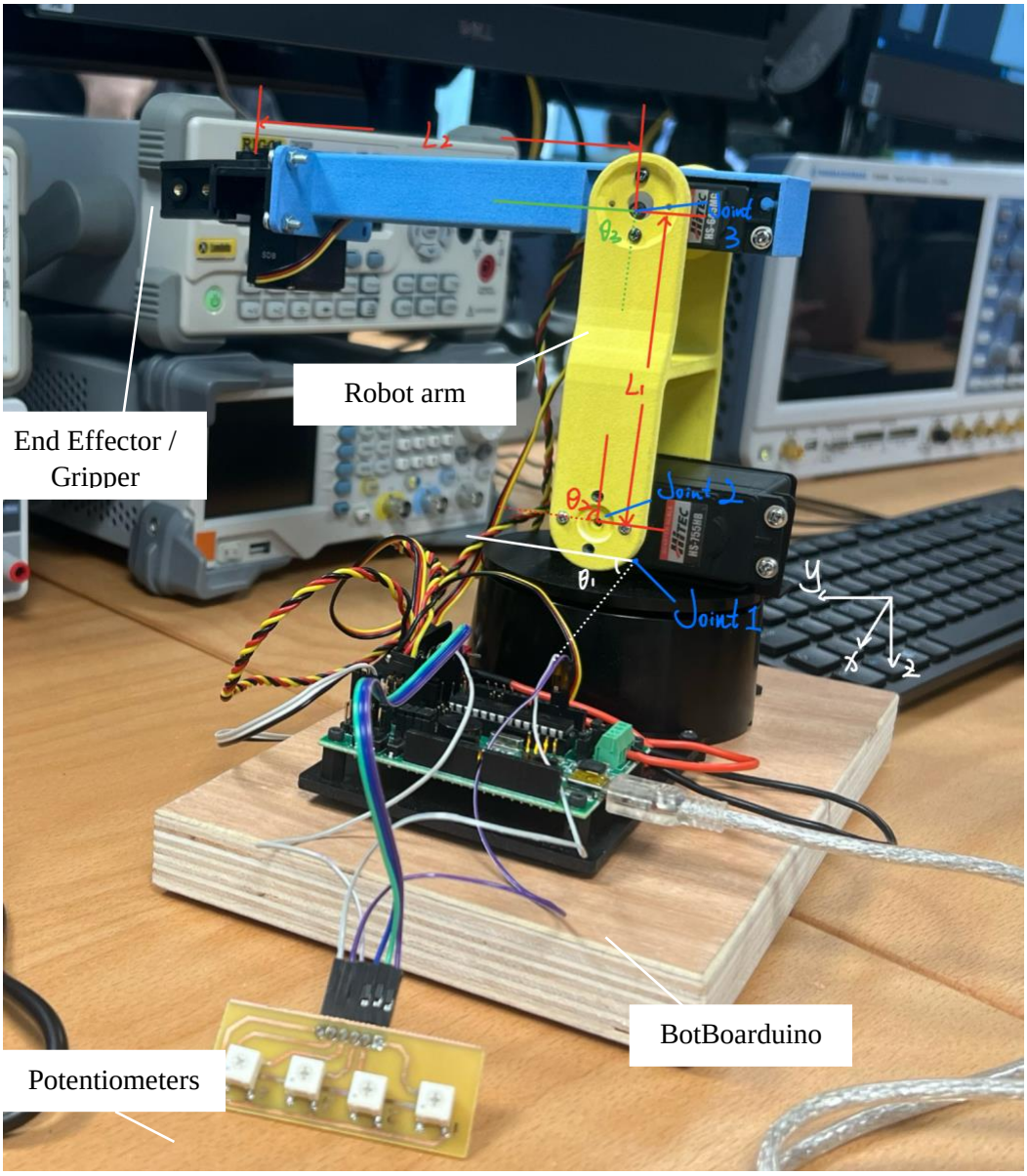

Figure 1:Photograph of the robot

Part 2 Forward Kinematics

We are required to find the position of the end-effector with respect to the base frame of the robot. To do so we need to first find the position of the last frame, frame {3} with respect to the base frame, frame {0} using compound transform. The position of the end-effector is then found by multiplying the position of the end-effector with respect to frame {3} to the compound transform.

The following axis diagram is drawn using the process of assigning Denavit-Hartenberg (DH) frames to a robotic arm. The mutually perpendicular lines MP_{12} and MP_{23} are drawn along the perpendiculars of Joint 1 to Joint 2 and Joint 2 to Joint 3 respectively. Then, the frames {1} to {n-1} (n-1=2, in this case), are drawn with Z_i pointing along the ith axis joint, X_i pointing along the mutual perpendicular, and Y_i based on the right-hand rule system. Frame {0} is drawn to match frame {1} and frame {n} is assigned the same way as the others, except X_i is pointing in the direction such that the nth joint parameter (in this case, θ_3) is 0.

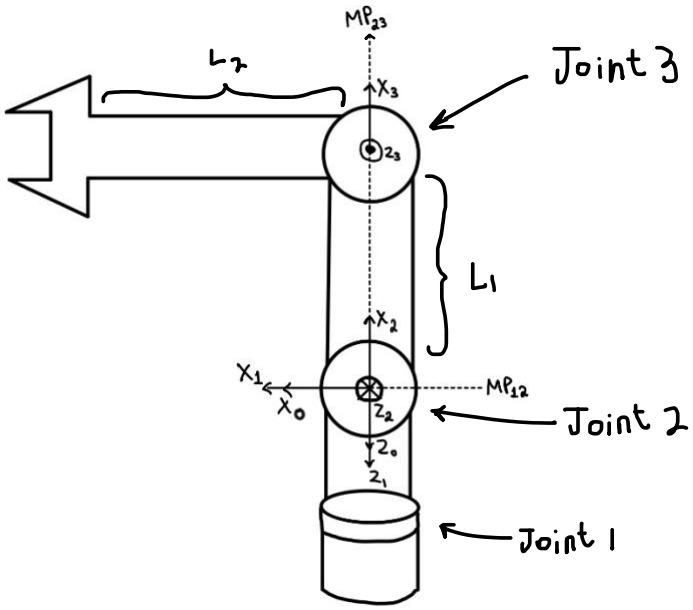

Figure 2:Diagram of the robot's axes and link lengths.

Compound Transform:
The general formula for the homogeneous transformation matrix is given by:

$$^{i-1}_{i}T = R_x(a_{i-1}) * D_x(a_{i-1}) * R_z(\theta_i) * D_z(d_i)$$

The Denavit-Hartenberg (DH) Parameters table below contains all the information required to describe the frame transformation (compound transform); 3 position parameters, and 3 rotation parameters.
Where:

- Link lengths: a_{i-1}, distance from $\hat{Z}_{i-1}$ to $\hat{Z}_i$ along $\hat{X}_{i-1}$. This is the distance between the two joints.
- Link twists: α_{i-1}, angle between $\hat{Z}_{i-1}$ to $\hat{Z}_i$ about $\hat{X}_{i-1}$. This is the angle between the two joint axes of the link.
- Link offsets: d_i, distance from $\hat{X}_{i-1}$ to $\hat{X}_i$ along $\hat{Z}_i$.

- Joint angles: θ_i, angle between $\hat{X}_{i-1}$ to $\hat{X}_i$ along $\hat{Z}_i$.

The parameters are found by observing the diagram of the robot and its axes. Observe below the DH table with each of the link lengths, twists, offsets, and joint angles.

i	a_{i-1}	a_{i-1}	d_i	θ_i
1	0°	0	0	θ_1
2	-90°	0	0	θ_2
3	180°	L_1	0	θ_3

Calculations

Joint angles of the robot are listed as:

$\theta_1 = \theta_1$ (variable)
$\theta_2 = \theta_2$ (variable)
$\theta_3 = \theta_3$ (variable)

The transition matrices are expressed as:

$$ {}^0_1T = \begin{bmatrix} c\theta_1 & -s\theta_1 & 0 & 0 \\ s\theta_1 & c\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, $$

$$ {}^1_2T = \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s\theta_2 & -c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, $$

$$ {}^2_3T = \begin{bmatrix} c\theta_3 & -s\theta_3 & 0 & L_1 \\ -s\theta_3 & -c\theta_3 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

With compound transform, it is known that:

$$ {}^0_3T = {}^0_1T \cdot {}^1_2T \cdot {}^2_3T $$

Hence, the transformation is calculated as the following. First, finding ${}^{0}_{2}T$ by conducting a compound transform on ${}^{0}_{1}T \cdot {}^{1}_{2}T$:

$$
{}^{0}_{2}T = {}^{0}_{1}T \cdot {}^{1}_{2}T = \begin{bmatrix} c\theta_1 & -s\theta_1 & 0 & 0 \\ s\theta_1 & c\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s\theta_2 & -c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} c\theta_1 c\theta_2 & -c\theta_1 s\theta_2 & -s\theta_1 & 0 \\ c\theta_1 s\theta_2 & -s\theta_1 s\theta_2 & c\theta_1 & 0 \\ -s\theta_2 & -c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Next, finding ${}^{0}_{2}T$ by conducting a compound transform on ${}^{0}_{1}T \cdot {}^{1}_{2}T \cdot {}^{2}_{3}T$:

$$
{}^{0}_{3}T = {}^{0}_{2}T \cdot {}^{2}_{3}T = {}^{0}_{1}T \cdot {}^{1}_{2}T \cdot {}^{2}_{3}T = \begin{bmatrix} c\theta_1 c\theta_2 & -c\theta_1 s\theta_2 & -s\theta_1 & 0 \\ c\theta_1 s\theta_2 & -s\theta_1 s\theta_2 & c\theta_1 & 0 \\ -s\theta_2 & -c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} c\theta_3 & -s\theta_3 & 0 & L_1 \\ -s\theta_3 & -c\theta_3 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} c\theta_1 c\theta_2 c\theta_3 + c\theta_1 s\theta_2 s\theta_3 & -c\theta_1 c\theta_2 s\theta_3 + c\theta_1 s\theta_2 c\theta_3 & s\theta_1 & L_1 c\theta_1 c\theta_2 \\ s\theta_1 s\theta_2 s\theta_3 + s\theta_1 c\theta_2 c\theta_3 & -s\theta_1 c\theta_2 s\theta_3 + s\theta_1 s\theta_2 c\theta_3 & -c\theta_1 & L_1 s\theta_1 c\theta_2 \\ -s\theta_2 c\theta_3 + c\theta_2 s\theta_3 & s\theta_2 s\theta_3 + c\theta_2 c\theta_3 & 0 & -L_1 s\theta_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Calculate the position and orientation of end-effector, we have:

$$
{}^{0}P = {}^{0}_{3}T \cdot {}^{3}P = \begin{bmatrix} c\theta_1 c\theta_2 c\theta_3 + c\theta_1 s\theta_2 s\theta_3 & -c\theta_1 c\theta_2 s\theta_3 + c\theta_1 s\theta_2 c\theta_3 & s\theta_1 & L_1 c\theta_1 c\theta_2 \\ s\theta_1 s\theta_2 s\theta_3 + s\theta_1 c\theta_2 c\theta_3 & -s\theta_1 c\theta_2 s\theta_3 + s\theta_1 s\theta_2 c\theta_3 & -c\theta_1 & L_1 s\theta_1 c\theta_2 \\ -s\theta_2 c\theta_3 + c\theta_2 s\theta_3 & s\theta_2 s\theta_3 + c\theta_2 c\theta_3 & 0 & -L_1 s\theta_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} L_2 \\ 0 \\ 0 \\ 1 \end{bmatrix}
$$

Finally, this gives us the vector of the **end-effector Cartesian position** (with respect to joint angles and link lengths):

$$
= \begin{bmatrix} L_2(c\theta_1 c\theta_2 c\theta_3 + c\theta_1 s\theta_2 s\theta_3) + L_1 c\theta_1 c\theta_2 \\ L_2(s\theta_1 s\theta_2 s\theta_3 + s\theta_1 c\theta_2 c\theta_3) + L_1 s\theta_1 c\theta_2 \\ L_2(c\theta_2 s\theta_3 - s\theta_2 c\theta_3) - L_1 s\theta_2 \\ 1 \end{bmatrix}
$$

Experimental Results

We tested this equation by writing Arduino code to set the angles of the joints of the BotBoarduino, and the following are the results:

Figure 3:(θ_1=0º, θ_2=90º, θ_3=0º)

Figure 4:(θ_1=0º, θ_2=90º, θ_3=90º)

Figure 5:(θ_1=0º, θ_2=0º, θ_3=0º)

Figure 6:(θ_1=30º, θ_2=60º, θ_3=90º)

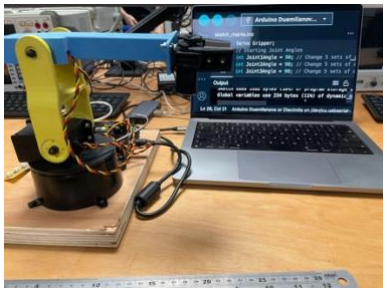

Figure 7:(θ_1=90º, θ_2=90º, θ_3=90º)

Hence, this verifies the equation obtained, because the orientations of the robot match the values provided by the Arduino code (note the computer with joint values next to the robot). Refer to Figure 2 to confirm that these angles and orientations correspond with one another. In the beginning of completing Forward Kinematics there were some minor errors in the orientation of the robot (i.e. not being exactly 90º), however, by setting the appropriate offsets we managed to have similar calculated results to experimental results.

Part 3 Inverse Kinematics

In this section, our goal is to find formulae for each of the joint angles (1,2,3), with respect to: x, y, z positions as well as L_1 and L_2. There are two primary ways to do this: geometric and algebraic methods. There is no singular deterministic process for inverse kinematics, in the way there is for forward kinematics, making the process more complex. In our case, we have chosen the geometric method, which involves analyzing the angles, distances, and lengths visually, and using trigonometric identities to find formulae to describe the angles.

Geometric Method

The geometric method for calculating inverse kinematic is as follows. The top-view of the robot can be seen below.

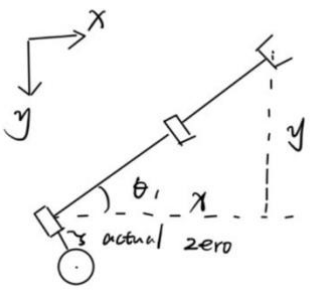

Figure 8: Top view drawing of robot

From this, we can obtain the following based on simple trigonometry:

$$\theta_1 = atan2(y, x)$$

This allows us to simplify the model to a 2D-problem, by ignoring the dimension that is created from θ_1. This creates the following diagram (the front-view of the robot):

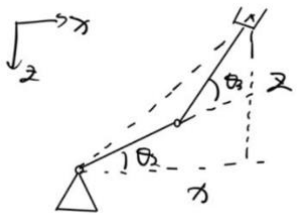

Figure 9: Front view drawing of robot

Based on this diagram, it is possible to apply the cosine rule:

$$x^2 + z^2 = {L_1}^2 + {L_2}^2 - 2L_1L_2\cos(180 - \theta_3)$$

$$= {L_1}^2 + {L_2}^2 + 2L_1L_2\cos(\theta_3)$$

Hence:

$$\theta_3 = \arccos\left(\frac{x^2 + z^2 - {L_1}^2 - {L_2}^2}{2L_1L_2}\right)$$

This is for the **elbow-down** case, but for the **elbow-up** case, we have:

$$\theta_3' = -\theta_3$$

Next, to solve for θ_2, the following diagram illustrates the same as before, but the angles have been split into ψ and β.

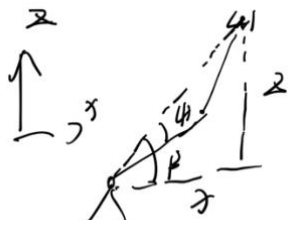

Figure 10: Front view drawing of robot, with β and ψ

$$\beta = atan2(z, x)$$

$$\cos(\psi) = \frac{-x^2 + z^2 + {L_1}^2 - {L_2}^2}{2L_1\sqrt{x^2 + z^2}}$$

In the **elbow-down case**:

$$\theta_2 = \beta - \psi$$

In the **elbow-up case**:

$$\theta_2' = \beta + \psi$$

Thus, we have obtained the formula to calculate the values (based on x, y, z input positions and link lengths) for θ_1 as well as the elbow up and elbow down cases for both θ_2 and θ_3.

Multiplicities

As shown above, there are multiple solutions available (like elbow up and elbow down). However, it is also important to note that, if $\theta_1 = 0$ or $\theta_1 = 180$, then instead of 2 multiplicities, there are 4 multiplicities; if the original 2 multiplicities (elbow up and elbow down) are [0, θ_2, θ_3] and [0, θ_2', θ_3'] (given that $\theta_1 = 0$), then the extra 2 multiplicities are [180, $180 - \theta_2$, $180 - \theta_3$] and [180, $180 - \theta_2'$, $180 - \theta_3'$]. Note that the converse is true if $\theta_1 = 180$ originally.

Experimental Results

We implemented this code in Arduino and tested it by measuring the position of the end-effector to make sure it matched the position inputted into the Arduino program:

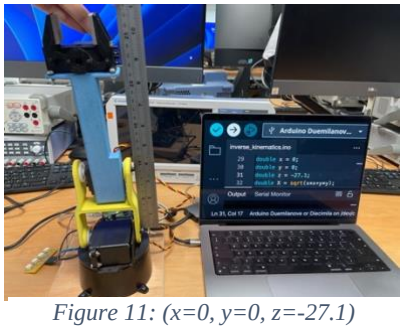

Figure 11: (x=0, y=0, z=-27.1)

Figure 12: (x=0, y=27.1, z=0)

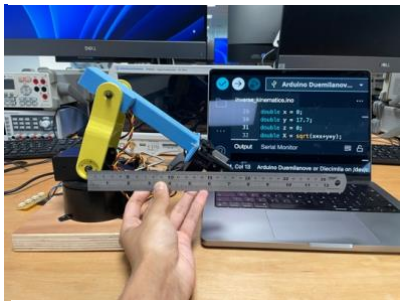

Figure 13: (x=0, y=17.7, z=0)

Figure 14: (x=0, y=15, z=-7)

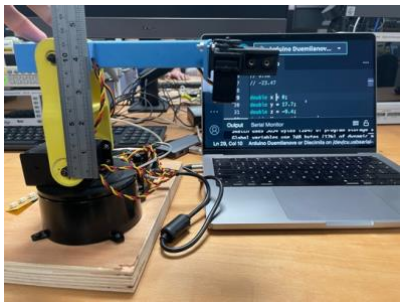

Figure 15: (x=0, y=-17.7, z=9.4)
(z measurement)

Figure 16: (x=0, y=-17.7, z=9.4)
(x measurement)

This verifies the equations that were derived for each of the joint angles since the position of the robot's end-effector matches the values provided by the Arduino code in each instance. However, it is important to note that our measurements of the link lengths were to the nearest millimeter, which means that there could be an error in the measurements of up to 0.1cm.

Summary

If the target point lies within the manipulator's workspace, there exists at least one set of solution to the joint angles. The workspace is defined as the volume of space the end-effector can reach.

Since we have a two-link robot with $L_1 \neq L_2$, the workspace is equal to the ring of radius $L_1 + L_2$ and the inner radius $|L_1 - L_2|$.

In our robot we have two possible solutions to achieve a particular position and orientation (elbow up or elbow down). The choice is ours to make and it depends on any possible obstacles in any of the solutions or possibility of damaging the robot due to object's weight.

Part 4 Trajectory Planning

Trajectory planning provides us with the time profile of position, velocity and acceleration of a movement. In terms of the robot, we built and are controlling, we can create a trajectory plan of a movement of our choice. In this case the robot moves from an initial position towards an object. However, the desired end point of the robot is required to be in its workspace so that the robot can reach that point. Also, the robot must not reach a point causing singularity since the robot may experience a loss in one of its Degrees of Freedom (DoF) or wear and tear.

There are three different trajectory plans we studied in this course, Cubic polynomial, Quintic polynomial, and Linear function with Parabolic blends. We chose to work with the cubic polynomial. The cubic polynomial contains the least constraints (4 constraints) which are required to be satisfied, thus, requires the least parameters ($u(0) = u_0$, $u(t_f) = u_f$, $\dot{u}(0) = 0$ & $\dot{u}(t_f) = 0$).

- $u(0) = u_0$, suggests that the initial position is u_0.
- , $u(t_f) = u_f$, t_f is the time to reach final position, therefore, this suggests u_f is the final position.
- $\dot{u}(0) = 0$, the velocity at the initial position is zero.
- $\dot{u}(t_f) = 0$, the velocity at the final position is zero (the cubic polynomial ensures that the start and end velocities are zero).

The cubic polynomial has the following structure:

$$u(t) = \begin{cases} a_0 + a_1 t + a_2 t^2 + a_3 t^3, & t \leq t_f \\ u_f, & t > t_f \end{cases}$$

We are told the initial position is such that Link 2 is up and Link 3 is forward. This initial position suggests that the joint angles are ($\theta_1 = 90°, \theta_2 = 90°, \theta_3 = 90°$). We chose the final position to have the following joint angles: ($\theta_1 = 45°, \theta_2 = 60°, \theta_3 = 0°$) with a final time of $t_f = 5s$ from initial position to final position.

By utilising the code, we made for forward kinematics we can calculate the (x, y, z) points corresponding to these angles.

The initial position, u_0, is equal to: $(0, 17.7, -9.4)$

The initial position, u_f, is equal to: $(9.58, 9.58, -23.47)$

We first begin by calculating the time profile of the robot's movement in terms of the x-axis.

$x_0 = 0, \;\; x_f = 9.58, \;\; t_f = 5s$:

We now utilise the parameters mentioned above:

$$x(0) = a_0 + a_1(0) + a_2(0)^2 + a_3(0)^3 = a_0 = x_0$$
$$x(t_f) = a_0 + a_1 t_f + a_2 {t_f}^2 + a_3 {t_f}^3 = x_f$$
$$\dot{u}(0) = a_1 + 2a_2(0) + 3a_3(0)^2 = a_1 = 0$$
$$\dot{u}(t_f) = a_1 + 2a_2 t_f + 3a_3 {t_f}^2 = 0$$

By following with further calculations, we can find the polynomial values a_2 and a_3:

$$a_2 = \frac{3}{{t_f}^2}(x_f - x_0)$$
$$a_3 = \frac{-2}{{t_f}^3}(x_f - x_0)$$

Utilising the initial and final position of the end-effector of the robot and the equations found to calculate the polynomial values using the 4 constraints we were able to express the time profile of the

robot for position, velocity, and acceleration for the specific movement it performed as a function of time. The MATLAB plots of the three-time profiles are shown below:

Observing the x, y, z cubic plots we can see a smooth transition from rest to movement and vice versa by looking at the start and end positions of the position time profiles. The velocity increase and decrease occur smoothly as well; the highest speed observed was about 2.5 cm/s. However, for the acceleration, in the x-axis the robot begins to decelerate from 2.3 cm/s^2, -1.87 cm/s^2 in the y-axis and -3.24 cm/s^2 in the z-axis. The acceleration time profile in the cubic polynomial raises many concerns for the robot. Due to the low number of constraints, the acceleration at the beginning and end of the trajectory suddenly jumps. This is an unrealistic trajectory for the robot and may cause jerky motions leading to potential problems for the robot.
To have control over the acceleration we could use the Quintic polynomial which would require two additional constraints (i.e. $\ddot{u}(0)$ & $\ddot{u}(t_f)$).

Knowing the final and initial positions of the robot, we can utilise inverse kinematics to send to

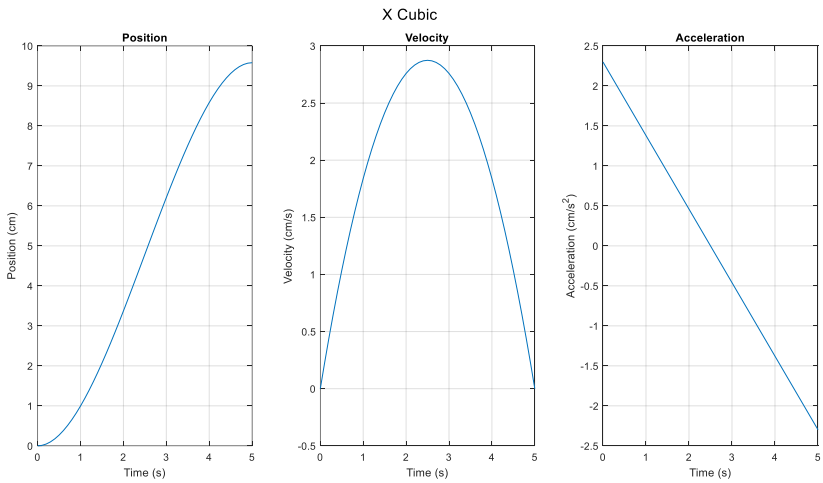

Figure 17: Position, Velocity, and Accelerations Graphs for X Trajectory

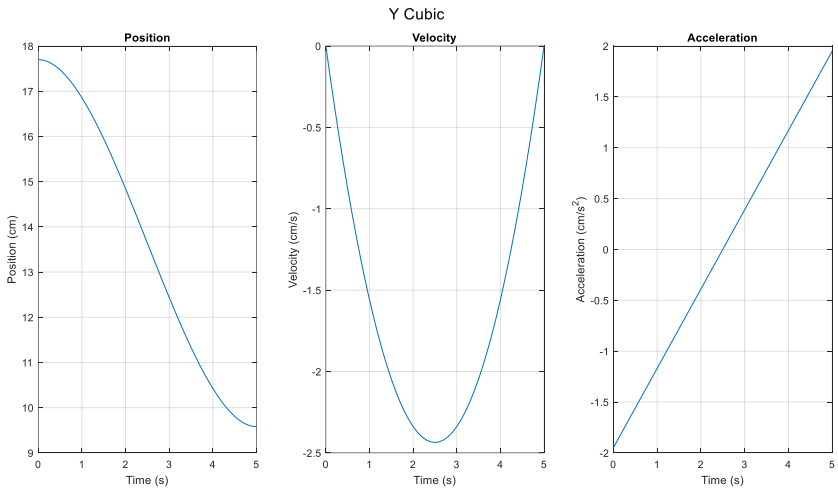

Figure 18: Position, Velocity, and Accelerations Graphs for Y Trajectory

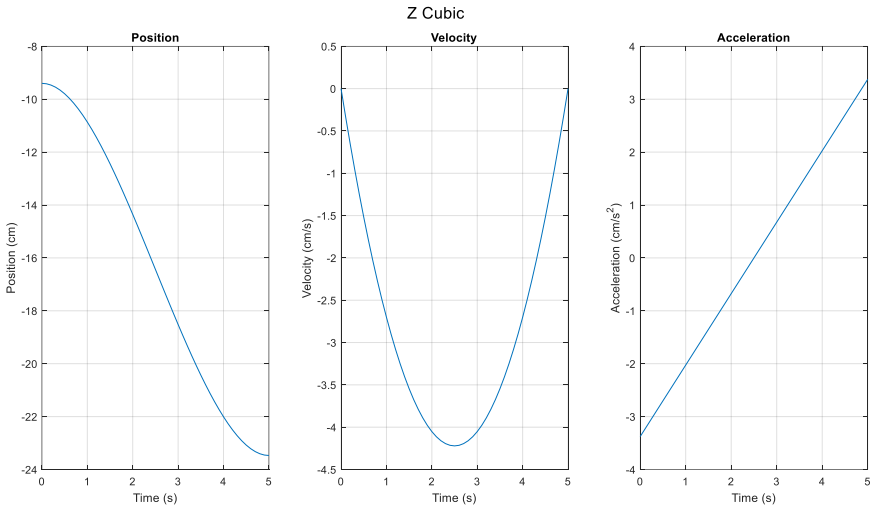

Figure 19: Position, Velocity, and Accelerations Graphs for Z Trajectory

Arduino the angles necessary for the joints and Trajectory planning to get each position necessary between the initial and final positions. At each position calculated using trajectory planning, angles are calculated for Arduino to send to the robot.

Experimental Results

The following images show that the trajectory planning was implemented in Arduino code, and experimentally produced a smooth, straight-line motion.

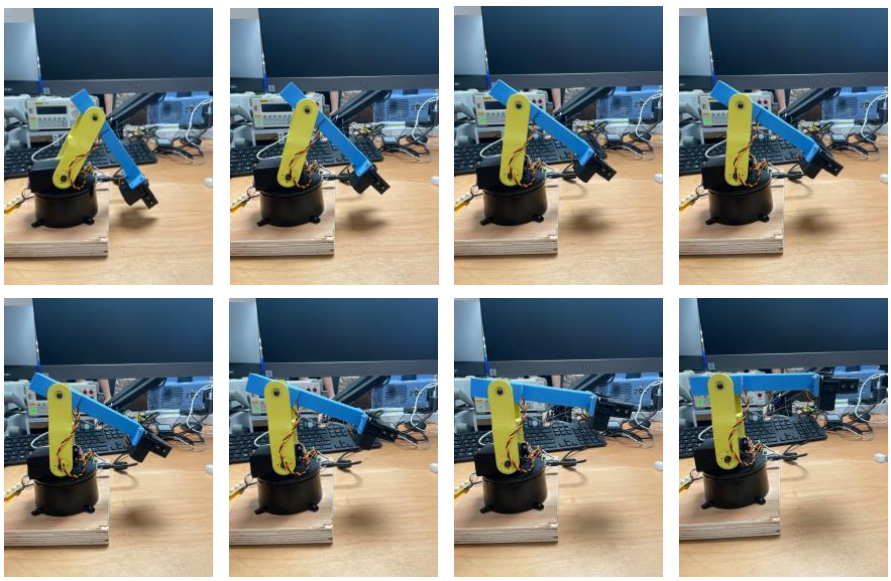

Figure 20: Trajectory planning straight-line movement

Conclusion

In this project, a robot arm with multiple functions is well designed and assembled. The achievement is mainly based on the theory of forward and inverse kinematics and trajectory planning. Additionally, the Arduino code for the “pick and place” task is provided as an Arduino file, and in the Appendix.

As described in the report above, the robot arm can achieve precise user and mechanical interaction through control code, achieving the “picking up” task, smooth linear motion and smooth placement of objects within the workspace. In our measurements, the error of the object grabbing and releasing position is controlled within a very reasonable range (minimum 0cm, maximum 0.75cm, typical value is 0.5cm). The reason for this is that it is impossible to perfectly measure the link lengths of the robot L_1 and L_2. Therefore, since we conduct inverse kinematics multiple times when planning the trajectory, it results in a propagation of error from one point to the next. Overall, this will result in an accumulated error in the final position when the robot is being placed. However, since this error is within a centimetre, it is almost negligible depending on the task and the size of the object.

In order to use the Pick and Place Arduino code, users can simply input the coordinates of any targets involved in the movement (home, ‘pick’, ‘place’, etc.), and it will be able to perform the movement as necessary. This indicates that all tasks are performed perfectly.

Appendix

The code for controlling picking up and dropping down:

```
#include <Servo.h>
#include <math.h>

// Arm Servo pins
#define Joint1Pin 2
#define Joint2Pin 3
#define Joint3Pin 4
#define GripperPin 11

//24 y, 7 x, 9 z
// Servo Objects
Servo Joint1;
Servo Joint2;
Servo Joint3;
Servo Gripper;

// Global variables for trajectory and arm dimensions
// MODIFY x0,y0 FOR START POSITION
// MODIFY xf,yf FOR END POSITION
// MODIFY z0 for HEIGHT PLACED DOWN
// MODIFY object_height
// MODIFY n

double object_height = 1.5;
double n = 3.0;
double ta = 5.0, ts = 10.0, tm = 15.0, tf = 20.0; // Total time for one-way trip
double x0 = 1.6, xa = x0, xh = 0, xf = -20; // X start, middle and end
```

```
double y0 = 15.7, ya = y0, yh = 17.7, yf = 7; // Y start, middle and end
double z0 = 8, za = z0 - n*object_height, zh = -9.4, zf = z0; // Z start, middle and end

double GripperOpen = 40; // Open gripper; Need to tune value
double GripperClose = 140; // Close gripper; Need to tune value

// Arm dimensions
const double L1 = 9.4;
const double L2 = 17.7;

// Offsets in degrees
double theta1Offset = 3; // Degrees
double theta2Offset = 3; // Degrees
double theta3Offset = 35; // Degrees

void trajectoryPlanning(double t, double &x, double &y, double &z, double tStart, double tEnd,
double xs, double ys, double zs, double xt, double yt, double zt);
void inverseKinematics(double x, double y, double z, double &theta1_deg, double &theta2_deg,
double &theta3_deg);

void setup() {
  Serial.begin(9600);
  Joint1.attach(Joint1Pin);
  Joint2.attach(Joint2Pin);
  Joint3.attach(Joint3Pin);
  Gripper.attach(GripperPin);

  // Move to middle position
  double middleTheta1, middleTheta2, middleTheta3;
  inverseKinematics(xh, yh, zh, middleTheta1, middleTheta2, middleTheta3);
  Joint1.write(int(middleTheta1 + theta1Offset));
  Joint2.write(int(middleTheta2 + theta2Offset));
  Joint3.write(int(middleTheta3 + theta3Offset));
  Gripper.write(int(GripperOpen));

  delay(3000); // Delay to ensure arm reaches middle position
}

void loop() {
    static bool taskCompleted = false;
    if (taskCompleted) return;

    // From home to point above start position
    for (int i = 0; i <= 10 * ts; i++) {
        double t = i * 0.1; // Simulate time progression
```

```
    double x, y, z;
    trajectoryPlanning(t, x, y, z, 0, ts, xh, yh, zh, xa, ya, za);

    double theta1_deg, theta2_deg, theta3_deg;
    inverseKinematics(x, y, z, theta1_deg, theta2_deg, theta3_deg);

    Joint1.write(int(theta1_deg + theta1Offset));
    Joint2.write(int(theta2_deg + theta2Offset));
    Joint3.write(int(theta3_deg + theta3Offset));

    delay(100);
}

// From point above start position to start position
for (int i = 0; i <= 10 * tm; i++) {
    double t = i * 0.1;
    double x, y, z;
    trajectoryPlanning(t, x, y, z, 0, tm, xa, ya, za, x0, y0, z0);

    double theta1_deg, theta2_deg, theta3_deg;
    inverseKinematics(x, y, z, theta1_deg, theta2_deg, theta3_deg);

    Joint1.write(int(theta1_deg + theta1Offset));
    Joint2.write(int(theta2_deg + theta2Offset));
    Joint3.write(int(theta3_deg + theta3Offset));

    delay(100);
}

  delay(3000); // Delay to ensure arm reaches start position

  Gripper.write(int(GripperClose)); // Close gripper to grabbing the object

  delay(3000);

// From t0 to tm
for (int i = 0; i <= 10 * tm; i++) {
    double t = i * 0.1;
    double x, y, z;
    trajectoryPlanning(t, x, y, z, 0, tm, x0, y0, z0, xh, yh, zh);

    double theta1_deg, theta2_deg, theta3_deg;
    inverseKinematics(x, y, z, theta1_deg, theta2_deg, theta3_deg);
```

```
        Joint1.write(int(theta1_deg + theta1Offset));
        Joint2.write(int(theta2_deg + theta2Offset));
        Joint3.write(int(theta3_deg + theta3Offset));

        delay(100);
    }

    // From tm to tf
    for (int i = 0; i <= 10 * (tf - tm); i++) {
        double t = tm + i * 0.1;
        double x, y, z;
        trajectoryPlanning(t, x, y, z, tm, tf, xh, yh, zh, xf, yf, zf);

        double theta1_deg, theta2_deg, theta3_deg;
        inverseKinematics(x, y, z, theta1_deg, theta2_deg, theta3_deg);

        Joint1.write(int(theta1_deg + theta1Offset));
        Joint2.write(int(theta2_deg + theta2Offset));
        Joint3.write(int(theta3_deg + theta3Offset));

        delay(100);
    }

    Gripper.write(int(GripperOpen)); // Open gripper to release the object

    taskCompleted = true;
}

void trajectoryPlanning(double t, double &x, double &y, double &z, double tStart, double tEnd,
double xs, double ys, double zs, double xt, double yt, double zt) {
    // Calculate trajectory based on current segment
    double dt = t - tStart; // Time since start of this segment
    double totalTime = tEnd - tStart; // Total time for this segment

    double a2 = 3 * (xt - xs) / pow(totalTime, 2);
    double a3 = -2 * (xt - xs) / pow(totalTime, 3);
    x = xs + a2 * pow(dt, 2) + a3 * pow(dt, 3);

    double b2 = 3 * (yt - ys) / pow(totalTime, 2);
    double b3 = -2 * (yt - ys) / pow(totalTime, 3);
    y = ys + b2 * pow(dt, 2) + b3 * pow(dt, 3);

    double c2 = 3 * (zt - zs) / pow(totalTime, 2);
    double c3 = -2 * (zt - zs) / pow(totalTime, 3);
```

```
    z = zs + c2 * pow(dt, 2) + c3 * pow(dt, 3);
}

void inverseKinematics(double x, double y, double z, double &theta1_deg, double &theta2_deg,
double &theta3_deg) {
  double X = sqrt(x*x + y*y);

  double theta1 = atan2(y, x);
  theta1_deg = theta1 * (180 / M_PI);

  double beta = atan2(z, X);
  double cos_psi = (L1*L1 + X*X + z*z - L2*L2) / (2 * L1 * sqrt(X*X + z*z));
  double psi = acos(cos_psi);
  double theta2 = -(beta - psi);
  theta2_deg = theta2 * (180 / M_PI);

  double theta3 = acos((X*X + z*z - L1*L1 - L2*L2) / (2 * L1 * L2));
  theta3_deg = theta3 * (180 / M_PI);
}
```